BULBASAUR

TEAM REFERENCE · ICPC

Capítulo 1: Strings · Edición 2026

CÓMO USAR ESTE CUADERNO

1. Copia la **plantilla base** en un archivo `main.cpp`.
2. Busca la receta según el problema en el **mapa rápido**.
3. Pega el `struct` o las funciones de la receta.
4. En `main`, lee los datos, llama a la función y muestra el resultado con `cout`.

No hace falta entender todo el código: las recetas ya están probadas. Cambia solamente lo que dice **Entrada** y **Salida**. Cada receta incluye un `main` de ejemplo; ese `main` se puede borrar cuando ya tengas el tuyo.

REGLA DE ORO

Si el problema pide posiciones y usa 1-based, imprime `pos + 1`. Todos los índices de este cuaderno empiezan en 0.

Si algo falla: revisa la lectura, el 0/1-based, el uso de `long long` y que el SAM reciba minúsculas.

MAPA RÁPIDO: ¿QUÉ NECESITO?

- ¿Aparece un patrón? **KMP**
- ¿Prefijos que también son sufijos? **KMP**
- ¿Patrón en texto, versión corta? **Z**
- ¿Palíndromos? **Manacher**
- ¿Comparar substrings rápido? **Hash**
- ¿Substrings distintas, ocurrencias, LCS? **SAM**

COMPILAR Y EJECUTAR

```
g++ -O2 -std=c++17 prog.cpp -o prog
./prog < entrada.txt > salida.txt
```

La primera línea compila; la segunda ejecuta. Si no usas archivos, ejecuta solamente `./prog`.

1 Strings

Un **string** es una palabra o secuencia de caracteres. En ICPC casi siempre se lee como una palabra sin espacios con `cin >> s`; si la entrada trae espacios, se lee con `getline`. Los algoritmos de este capítulo responden preguntas sobre substrings, prefijos, sufijos, palíndromos y repeticiones.

1.1 Plantilla base: todo programa empieza aquí

Sirve para: tener un programa que ya compila y ejecuta. Siempre se empieza desde aquí.

Qué cambiar: la parte de leer, la llamada a la receta y la parte de imprimir. El resto se deja igual.

Salida del ejemplo: el largo del string leído.

```
// PLANTILLA BASE. Copia este archivo y ejecutalo con:
// g++ -O2 -std=c++17 programa.cpp -o programa
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // 1) LEER. cin >> s lee UNA palabra (se detiene en un espacio).
    string s;
    cin >> s;

    // 2) RESOLVER. Pega aqui el struct/funcion de la receta.
    //    Luego llamalo con los datos leidos.
    int largo = (int)s.size();

    // 3) IMPRIMIR. Cambia esto por lo que pida el problema.
    cout << largo << '\n';
    return 0;
}
```

Ojo: Si el problema tiene varios casos, envuelve el cuerpo del `main` en un `while (t--)` y lee la cantidad de casos `t` al inicio. No mezcles `cin >>` con `getline` sin leer el problema de lectura en la receta siguiente.

1.2 Operaciones de string que ya existen

No hace falta programar lo básico: C++ ya trae estas operaciones.

Quiero...	Escribo...
leer una palabra	`cin >> s;`
leer una línea con espacios	`getline(cin, s);`
saber su largo	`s.size()`
ver el carácter i	`s[i]` (i empieza en 0)
copiar un pedazo	`s.substr(pos, len);`
buscar un texto	`s.find(t)`
agregar al final	`s += t;`
recorrer todos los caracteres	`for (char c : s)`
invertir	`reverse(s.begin(), s.end());`
ordenar	`sort(s.begin(), s.end());`

Leer una línea después de un número

`cin >> n` deja el salto de línea pendiente. Antes de `getline` hay que consumirlo:

```
int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
string linea;
getline(cin, linea);  // ahora si lee la linea completa
```

Ojo: `s.find(t)` devuelve `string::npos` cuando no encuentra `t`. Las posiciones que devuelven `find`, KMP y Z son 0-based. Si el problema numera desde 1, imprime `pos + 1`.

1.3 KMP: buscar un patrón

Sirve para: encontrar todas las apariciones de un patrón dentro de un texto, incluso si se solapan. También calcula prefijos que son sufijos.

Entrada: dos palabras sin espacios: `texto` y `patron`.

Salida: las posiciones donde empieza el patrón, contando desde 0.

Complejidad: O(n + m), con n = largo del texto y m = largo del patrón.

```
// KMP: busca TODAS las apariciones de un patron dentro de un texto.
// Entrada: texto patron (palabras sin espacios).
// Salida: cantidad de apariciones y sus posiciones (0 = primero).
#include <bits/stdc++.h>
using namespace std;

// pi[i] = largo del prefijo mas largo de s que tambien es sufijo
// de s[0..i]. Ejemplo: "ababa" -> [0,0,1,2,3].
vector<int> prefix_function(const string &s) {
    int n = (int)s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

// Devuelve las posiciones donde empieza el patron dentro del texto.
vector<int> kmp_search(const string &text, const string &pat) {
```

```
    if (pat.empty()) return {};
    vector<int> pi = prefix_function(pat), pos;
    int j = 0; // cuantos caracteres del patron ya coinciden
    for (int i = 0; i < (int)text.size(); ++i) {
        while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
        if (text[i] == pat[j]) ++j;
        if (j == (int)pat.size()) {
            pos.push_back(i - (int)pat.size() + 1);
            j = pi[j - 1]; // permite apariciones solapadas
        }
    }
    return pos;
}

int main() {
    string text, pat;
    cin >> text >> pat;

    vector<int> pos = kmp_search(text, pat);
    cout << pos.size() << '\n';
    for (int i = 0; i < (int)pos.size(); ++i) {
        if (i) cout << ' ';
        cout << pos[i];
    }
    cout << '\n';
    return 0;
}
```

Qué significa `pi`

`pi[i]` es el largo del prefijo más largo de `pat[0..i]` que también es sufijo de ese mismo pedazo. Por ejemplo, para `"ababa"`:

$$\texttt{pi} = [0,\ 0,\ 1,\ 2,\ 3].$$

El último valor, `pi[m-1]`, es el borde más largo del patrón. El período más pequeño es `m - pi[m-1]`. Si el período divide a `m`, el patrón está formado por repeticiones de ese bloque.

Ojo: Si solo necesitas saber si el patrón aparece una vez, también puedes usar `s.find(pat) != string::npos`. KMP conviene cuando hay muchos patrones, textos largos o se piden todas las posiciones.

1.4 Z-function: prefijos repetidos

Sirve para: saber, para cada posición, cuántos caracteres coinciden con el comienzo del string. Es una alternativa corta a KMP.

Entrada: dos palabras sin espacios: `texto` y `patron`.

Salida: posiciones 0-based donde aparece el patrón.

Complejidad: O(n + m).

```
// Z-FUNCTION: z[i] = largo del prefijo mas largo de s que
// coincide con s[i..]. Sirve para buscar patrones y para
// comparar prefijos/sufijos.
#include <bits/stdc++.h>
using namespace std;

vector<int> z_function(const string &s) {
    int n = (int)s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
    }
    return z;
}

int main() {
    string text, pat;
    cin >> text >> pat;
    if (pat.empty()) return 0;

    // '#' debe ser un caracter que NO aparezca en text ni en pat.
    string t = pat + "#" + text;
    vector<int> z = z_function(t);

    vector<int> pos;
    int start = (int)pat.size() + 1;
    for (int i = start; i < (int)t.size(); ++i) {
        if (z[i] == (int)pat.size()) pos.push_back(i - start);
    }

    cout << pos.size() << '\n';
    for (int i = 0; i < (int)pos.size(); ++i) {
        if (i) cout << ' ';
        cout << pos[i];
    }
    cout << '\n';
    return 0;
}
```

Qué guarda `z`

`z[i]` es el largo del prefijo más largo de `s` que coincide con `s[i..]`. Por convención, `z[0] = 0`. Para buscar un patrón se construye:

$$\texttt{t = patron + '\#' + texto.}$$

En las posiciones del texto, `z[i] == m` significa que ahí empieza una aparición completa del patrón.

Cómo usarlo en tu `main`

```
string t = patron + "#" + texto;  // '#' no debe estar en las strings
vector<int> z = z_function(t);
for (int i = (int)patron.size() + 1; i < (int)t.size(); ++i)
    if (z[i] == (int)patron.size()) {
        int pos = i - (int)patron.size() - 1;
        cout << pos << '\n';
    }
```

Ojo: Si el string puede contener cualquier carácter, elige un separador que no aparezca (por ejemplo `'$'` o un carácter especial). Si no, usa KMP. El separador debe ser distinto de todos los caracteres de entrada.

1.5 Manacher: palíndromos en O(n)

Sirve para: encontrar el palíndromo más largo, contar palíndromos o responder preguntas sobre palíndromos centrados en una posición.

Entrada: una palabra sin espacios.

Salida: el palíndromo más largo, su largo y la cantidad total de substrings palindrómicas (contando repetidas).

Complejidad: O(n).

```
// MANACHER: todos los palindromos en O(n).
// d1[i] = radio impar: s[i-k..i+k] es palindromo.
// d2[i] = radio par:   s[i-k..i+k-1] es palindromo.
#include <bits/stdc++.h>
using namespace std;

struct Manacher {
    vector<int> d1, d2;

    Manacher(const string &s) {
        int n = (int)s.size();
        d1.assign(n, 0);
        d2.assign(n, 0);

        // Palindromos de largo impar, centro en i.
        for (int i = 0, l = 0, r = -1; i < n; ++i) {
            int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
            while (i - k >= 0 && i + k < n &&
                   s[i - k] == s[i + k]) ++k;
            d1[i] = k--;
            if (i + k > r) { l = i - k; r = i + k; }
        }

        // Palindromos de largo par, centro entre i-1 e i.
        for (int i = 0, l = 0, r = -1; i < n; ++i) {
            int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
            while (i - k - 1 >= 0 && i + k < n &&
                   s[i - k - 1] == s[i + k]) ++k;
            d2[i] = k--;
            if (i + k > r) { l = i - k - 1; r = i + k; }
        }
    }

    // Cantidad de substrings palindromas (contando repetidas).
    long long count_all() const {
        long long ans = 0;
        for (int i = 0; i < (int)d1.size(); ++i) {
            ans += d1[i] + d2[i];
        }
        return ans;
    }

    // {inicio, largo} del palindromo mas largo.
    pair<int, int> longest() const {
        int best = 0, start = 0;
        for (int i = 0; i < (int)d1.size(); ++i) {
            int len = 2 * d1[i] - 1;
            if (len > best) { best = len; start = i - d1[i] + 1; }
            len = 2 * d2[i];
            if (len > best) { best = len; start = i - d2[i]; }
        }
        return {start, best};
    }
};

int main() {
    string s;
    cin >> s;

    Manacher m(s);
```

```
    pair<int, int> p = m.longest();
    cout << "Palindromo mas largo: "
         << s.substr(p.first, p.second) << '\n';
    cout << "Su largo: " << p.second << '\n';
    cout << "Palindromos contando repetidos: "
         << m.count_all() << '\n';
    return 0;
}
```

Cómo leer `d1` y `d2`

- `d1[i]`: radio de palíndromos de largo impar con centro en `i`. El largo es `2*d1[i] - 1` y el inicio es `i - d1[i] + 1`.
- `d2[i]`: radio de palíndromos de largo par centrados entre `i-1` e `i`. El largo es `2*d2[i]` y el inicio es `i - d2[i]`.

El struct ya trae dos atajos listos: `longest()` devuelve `{inicio, largo}` y `count_all()` suma todos los palíndromos contando apariciones repetidas.

Ojo: Si el problema pide palíndromos **distintos**, `count_all` no sirve directamente: hay que eliminar duplicados con un palindromic tree o con hashing. Para el más largo y para consultar centros, Manacher es exacto y muy rápido.

1.6 Hash doble: comparar substrings rápido

Sirve para: comparar dos pedazos de un string en O(1), calcular el largo del prefijo común (LCP) y buscar patrones con búsqueda binaria.

Entrada: un string `s`, una cantidad de consultas `q` y cada consulta con dos rangos `l1 r1 l2 r2`.

Salida: `SI` si `s[l1..r1)` es igual a `s[l2..r2)`, o `NO` si no.

Complejidad: Preprocesar O(n); cada comparación O(1); cada LCP O(log n).

```
// HASH DOBLE: compara substrings en O(1) y calcula LCP en O(log n).
// Es probabilistico: con dos modulos, el riesgo de colision
// es minimo. Entrada: s, q consultas l1 r1 l2 r2.
// Los rangos son [l, r), con 0 <= l < r <= n.
// Salida: SI si las dos substrings son iguales, NO si no.
#include <bits/stdc++.h>
using namespace std;

struct StringHash {
    static const long long M1 = 1000000007LL;
    static const long long M2 = 1000000009LL;
    static const long long B  = 911382323LL;

    int n;
    vector<long long> p1, p2, h1, h2;

    StringHash(const string &s)
        : n((int)s.size()), p1(n + 1, 1), p2(n + 1, 1),
          h1(n + 1, 0), h2(n + 1, 0) {
        for (int i = 0; i < n; ++i) {
            long long c = (unsigned char)s[i] + 1;
            p1[i + 1] = p1[i] * B % M1;
            p2[i + 1] = p2[i] * B % M2;
            h1[i + 1] = (h1[i] * B + c) % M1;
            h2[i + 1] = (h2[i] * B + c) % M2;
        }
    }

    // Hash de s[l..r). El rango incluye l y NO incluye r.
    pair<long long, long long> get(int l, int r) const {
        long long a = (h1[r] - h1[l] * p1[r - l]) % M1;
        long long b = (h2[r] - h2[l] * p2[r - l]) % M2;
        if (a < 0) a += M1;
        if (b < 0) b += M2;
        return {a, b};
    }

    bool equal(int l1, int r1, int l2, int r2) const {
        if (r1 - l1 != r2 - l2) return false;
        return get(l1, r1) == get(l2, r2);
    }

    // Largo del prefijo comun mas largo de los sufijos i y j.
    int lcp(int i, int j) const {
        int low = 0, high = min(n - i, n - j);
        while (low < high) {
            int mid = (low + high + 1) / 2;
            if (get(i, i + mid) == get(j, j + mid)) low = mid;
            else high = mid - 1;
        }
        return low;
    }
};

int main() {
    string s;
    cin >> s;
    StringHash h(s);
    int q;
    cin >> q;
    while (q--) {
        int l1, r1, l2, r2;
        cin >> l1 >> r1 >> l2 >> r2;
        cout << (h.equal(l1, r1, l2, r2) ? "SI" : "NO") << '\n';
    }
    return 0;
}
```

Los rangos son medios abiertos

`[l, r)` significa: incluye `l` y **no** incluye `r`. Por eso `s[l..r)` se escribe `s.substr(l, r - l)`. Un rango vacío es `l == r`.

Ojo: El hash es probabilístico: con dos módulos la probabilidad de error es prácticamente nula, pero no es cero. Si el problema exige certeza absoluta, usa un autómata de sufijos o compara los strings de verdad. No uses un solo módulo si puedes evitarlo.

1.7 Suffix Automaton: la estructura estrella

Sirve para: resolver casi cualquier problema de substrings en O(n): contarlas, contarlas distintas, buscar ocurrencias, obtener la substring común más larga y la k-ésima en orden lexicográfico.

Alfabeto: el código usa solo letras `a`–`z`. Si la entrada trae mayúsculas, conviértelas con `tolower` antes de construir.

Ejemplo de entrada: `s p k`, por ejemplo `ababa aba 3`.

Complejidad: construir O(n); cada consulta O(1) u O($|p|$); la k-ésima substring es O(largo_respuesta $\times 26$).

La idea sin teoría

El autómata es un grafo de estados. Cada camino desde el estado inicial forma un substring de `s`, y todos los substrings están representados sin guardarlos uno por uno. Los estados se agrupan por el conjunto de posiciones donde terminan sus substrings. Eso permite contar ocurrencias y responder consultas con muy poco código adicional.

- `len[v]`: largo del substring más largo del estado `v`.
- `link[v]`: enlace al estado del sufijo propio más largo.
- `next[v][c]`: a qué estado se va al agregar el carácter `c`.
- `occ[v]`: cuántas veces aparece cada substring de `v` en `s`. Se calcula automáticamente al construir.

Funciones que ya vienen listas

El problema pide...	Usa...
substrings distintas	`sam.distinct_substrings()`
ocurrencias de un patrón	`sam.count_occurrences(p)`
substring común más larga	`sam.longest_common_substring(p)`
k-ésima substring lexicográfica	`sam.kth_distinct(k)`
substrings con al menos k apariciones	`sam.distinct_with_min_occ(k)`

```
// SUFFIX AUTOMATON (AUTOMATA DE SUFIJOS).
// Construye en O(n) una maquina que reconoce todos los substrings.
// Solo usa letras 'a'..'z': si hay mayusculas, pasalas a minusculas.
// Entrada: s p k (k <= 10^18). Salida: varias respuestas de prueba.
#include <bits/stdc++.h>
using namespace std;

struct SuffixAutomaton {
    static const int ALPHA = 26;
    static const long long LIMIT = 4000000000000000000LL;

    struct State {
        int len, link;
        int next[ALPHA];
        State(int l = 0, int link_ = -1) : len(l), link(link_) {
            fill(next, next + ALPHA, -1);
        }
    };

    vector<State> st;       // estados del automata
    vector<int> occ;        // occ[v] = ocurrencias de sus substrings
    vector<long long> ways; // ways[v] = substrings distintos desde v
    int last;

    SuffixAutomaton(const string &s = "") {
        st.reserve(2 * max(1, (int)s.size()));
        st.push_back(State()); // estado inicial
        // el estado inicial no cuenta ocurrencias
        occ.push_back(0);
        last = 0;
```

```
    for (char ch : s) extend(ch);
    compute_occurrences();
    compute_ways();
}

// Agrega un caracter al final del string.
void extend(char ch) {
    int c = ch - 'a';
    int cur = (int)st.size();
    st.push_back(State(st[last].len + 1));
    occ.push_back(1); // este estado representa un prefijo real

    int p = last;
    while (p != -1 && st[p].next[c] == -1) {
        st[p].next[c] = cur;
        p = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = (int)st.size();
            st.push_back(st[q]); // copia transiciones y link
            st[clone].len = st[p].len + 1;
            occ.push_back(0); // un clon no es un prefijo real
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}

// Ordena los estados por len de menor a mayor.
vector<int> order_by_len() const {
    int n = (int)st.size();
    int max_len = st[last].len;
    vector<int> cnt(max_len + 1, 0), order(n);
    for (int i = 0; i < n; ++i) ++cnt[st[i].len];
    for (int i = 1; i <= max_len; ++i) cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; --i) {
        order[--cnt[st[i].len]] = i;
    }
    return order;
}

// Propaga las ocurrencias por los suffix links.
void compute_occurrences() {
    vector<int> order = order_by_len();
    for (int i = (int)order.size() - 1; i > 0; --i) {
        int v = order[i];
        if (st[v].link != -1) occ[st[v].link] += occ[v];
    }
}

// Cuenta los caminos (substrings distintos) desde cada estado.
void compute_ways() {
    int n = (int)st.size();
    ways.assign(n, 0);
    vector<int> order = order_by_len();
    for (int i = n - 1; i >= 0; --i) {
        int v = order[i];
        long long total = 0;
        for (int c = 0; c < ALPHA; ++c) {
            int u = st[v].next[c];
            if (u == -1) continue;
            // la arista sola + todas sus extensiones
            total += 1 + ways[u];
            if (total > LIMIT) { total = LIMIT; break; }
        }
        ways[v] = total;
    }
}

// Cantidad de substrings distintos (sin contar el string vacio).
long long distinct_substrings() const {
    long long ans = 0;
    for (int v = 1; v < (int)st.size(); ++v) {
        ans += st[v].len - st[st[v].link].len;
    }
    return ans;
}

// Cuantas veces aparece p en s. Devuelve 0 si no aparece.
int count_occurrences(const string &p) const {
    if (p.empty()) return 0;
    int v = 0;
    for (char ch : p) {
        int c = ch - 'a';
        if (c < 0 || c >= ALPHA || st[v].next[c] == -1) return 0;
        v = st[v].next[c];
    }
    return occ[v];
}

// Largo del substring comun mas largo entre s y t.
int longest_common_substring(const string &t) const {
    int v = 0, len = 0, best = 0;
    for (char ch : t) {
        int c = ch - 'a';
        if (c < 0 || c >= ALPHA) { v = 0; len = 0; continue; }
        while (v != -1 && st[v].next[c] == -1) v = st[v].link;
        if (v == -1) { v = 0; len = 0; continue; }
        len = min(len, st[v].len) + 1;
        v = st[v].next[c];
        best = max(best, len);
    }
    return best;
}

// k-esima substring distinta en orden lexicografico
// (k empieza en 1).
// Si no existe, devuelve "".
string kth_distinct(long long k) const {
    if (k <= 0 || k > ways[0]) return "";
    string ans;
    int v = 0;
    while (true) {
        for (int c = 0; c < ALPHA; ++c) {
            int u = st[v].next[c];
            if (u == -1) continue;
            // substrings que empiezan con esta letra
            long long cnt = 1 + ways[u];
            if (k > cnt) { k -= cnt; continue; }
            ans.push_back(char('a' + c));
            --k;
            if (k == 0) return ans;
            v = u;
            break;
        }
    }
}

// Cantidad de substrings distintos que aparecen al menos k veces.
long long distinct_with_min_occ(int k) const {
    if (k <= 0) return distinct_substrings();
    long long ans = 0;
    for (int v = 1; v < (int)st.size(); ++v) {
        if (occ[v] >= k) ans += st[v].len - st[st[v].link].len;
    }
    return ans;
}
};

int main() {
    string s, p;
    long long k;
    cin >> s >> p >> k;

    SuffixAutomaton sam(s);
    cout << "Distintas: " << sam.distinct_substrings() << '\n';
    cout << "Ocurrencias de p: " << sam.count_occurrences(p) << '\n';
    cout << "LCS con p: " << sam.longest_common_substring(p) << '\n';
    cout << "k-esima: " << sam.kth_distinct(k) << '\n';
    cout << "Con >=2 ocurrencias: "
         << sam.distinct_with_min_occ(2) << '\n';
    return 0;
}
```

Cómo usarlo en tu `main`

```
SuffixAutomaton sam(s);              // s en minusculas, una sola vez

long long total = sam.distinct_substrings();
int veces = sam.count_occurrences(patron);
int lcs = sam.longest_common_substring(otro);
string kth = sam.kth_distinct(k);    // k empieza en 1
long long repetidas = sam.distinct_with_min_occ(2);
```

Ejemplo del `main` incluido

Con la entrada `ababa aba 3`, el programa imprime:

```
Distintas: 9
Ocurrencias de p: 2
LCS con p: 3
k-esima: aba
Con >=2 ocurrencias: 5
```

Así puedes comprobar que copiaste todo bien. En el juez, cambia el `main` por el del problema: construye `sam` con el string grande y llama únicamente a las funciones que necesites.

Avisos importantes

- `kth_distinct` devuelve `""` si `k` no existe. Solo se garantiza hasta `k <= 10^18`.
- `occ` se calcula al construir. No hace falta llamar a ninguna función extra, pero no agregues caracteres después con `extend` sin reconstruir el autómata.
- Para varias consultas, construye el SAM **una vez** y reutilízalo; no lo construyas dentro de cada consulta.
- Si el alfabeto no es `a`–`z`, cambia `ALPHA` y la conversión `ch - 'a'`. Si `n` es enorme y el alfabeto es grande, usa una versión con `map<char,int>`.